SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 0606 **COURSE NAME:** MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning (BL 5)

Assessment Tool Weightage: 40%

QUESTIONS: 05 **Total Marks:**100

Annexure A

Coding Challenge

Code of Conduct:

I Savitha R / 192421368 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: SAVITHA R

Annexure A

Short Answer Test

1, Write a Python program to implement the K-Nearest Neighbour (KNN) algorithm for a small dataset. Use Euclidean distance to find nearest neighbors and classify a test instance. Allow the user to input the value of K and display predicted class labels with accuracy.

PROGRAM:

```
import math
# Training dataset
data = [
    ([1, 2], "A"),
    ([2, 3], "A"),
    ([3, 3], "A"),
    ([6, 5], "B"),
    ([7, 7], "B"),
    ([8, 6], "B")
]

# Test data
test_data = [
    ([2, 2], "A"),
    ([7, 6], "B"),
    ([4, 4], "A")
]

k = int(input("Enter the value of K: "))
```

```
def euclidean_distance(p1, p2):
    return math.sqrt(sum((a - b) ** 2 for a, b in zip(p1, p2)))

def predict(test_point):
    distances = []

    for features, label in data:
        distance = euclidean_distance(test_point, features)
        distances.append((distance, label))

    distances.sort()

    neighbors = distances[:k]

    votes = {}
    for distance, label in neighbors:
        votes[label] = votes.get(label, 0) + 1
    return max(votes, key=votes.get)

correct = 0

print("\nPredictions:")

for features, actual in test_data:
    predicted = predict(features)

    print("Test point:", features,
          "Actual:", actual,
          "Predicted:", predicted)
```

```
    if predicted == actual:
        correct += 1

accuracy = (correct / len(test_data)) * 100

print("\nAccuracy:", accuracy, "%")
```

OUTPUT:

```
=============== RESTART: E:/Machine Learning/C04-AT1 PROGRAM1.py =
Enter the value of K: 3

Predictions:
Test point: [2, 2] Actual: A Predicted: A
Test point: [7, 6] Actual: B Predicted: B
Test point: [4, 4] Actual: A Predicted: A

Accuracy: 100.0 %
```

2. Develop a Python program to implement Locally Weighted Regression. Given a dataset, compute weights based on distance and predict output for a query point. Use a simple kernel function and compare predictions with standard linear regression for better understanding.

PROGRAM:

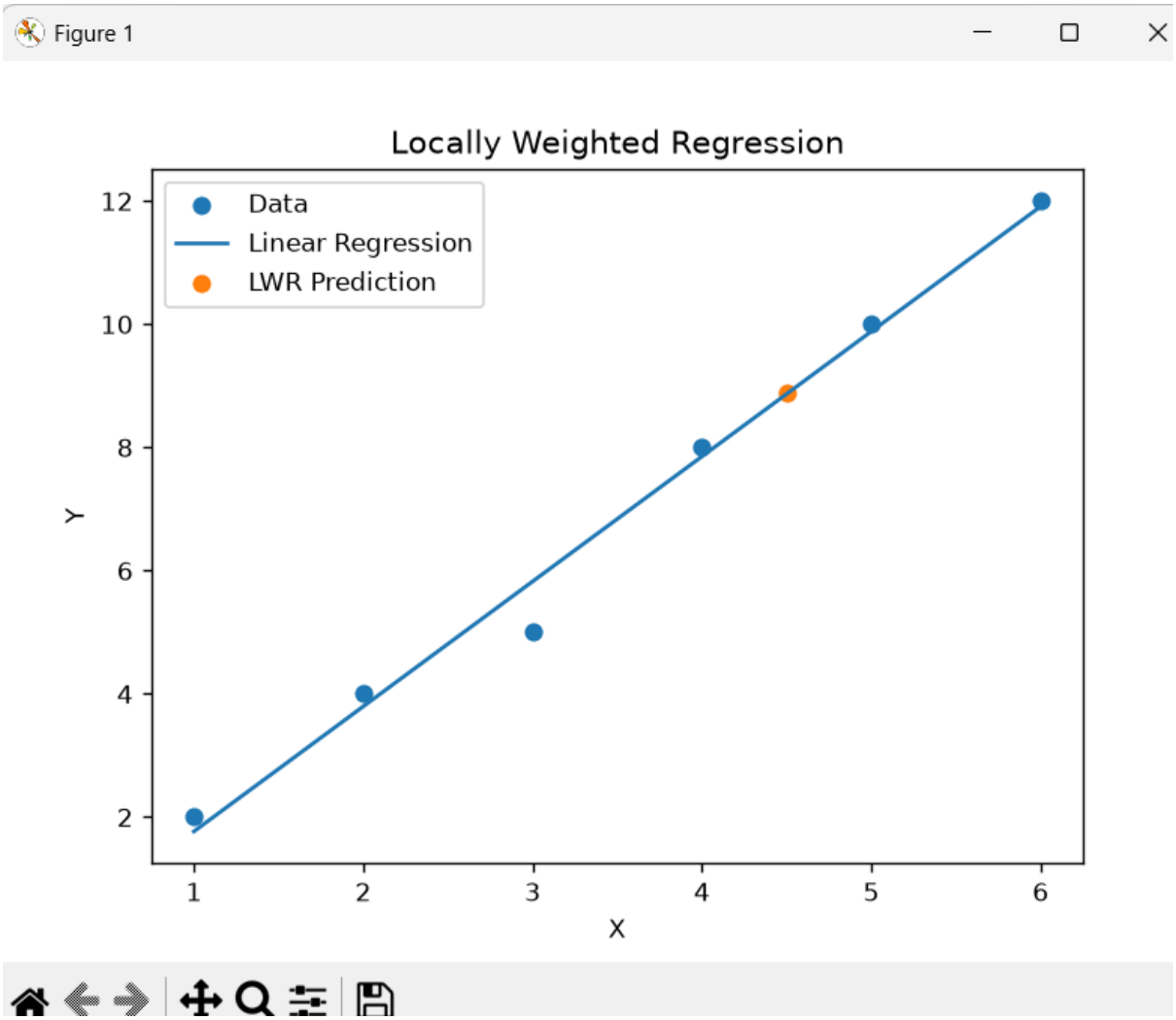
```
import numpy as np
import matplotlib.pyplot as plt

# Dataset
X = np.array([1, 2, 3, 4, 5, 6])
Y = np.array([2, 4, 5, 8, 10, 12])

# Query point
```

```
query = float(input("Enter query point: "))

# Bandwidth
tau = 1.0

def locally_weighted_regression(x_query):
    weights = np.exp(-((X - x_query) ** 2) / (2 * tau ** 2))

    X_matrix = np.column_stack((np.ones(len(X)), X))

    W = np.diag(weights)

    theta = np.linalg.inv(
        X_matrix.T @ W @ X_matrix
    ) @ (X_matrix.T @ W @ Y)

    prediction = np.array([1, x_query]) @ theta

    return prediction

# LWR prediction
lwr_prediction = locally_weighted_regression(query)

# Standard linear regression
coefficients = np.polyfit(X, Y, 1)
linear_prediction = np.polyval(coefficients, query)

print("\nLocally Weighted Regression Prediction:",
      lwr_prediction)
```

```
print("Standard Linear Regression Prediction:",
      linear_prediction)

# Visualization
plt.scatter(X, Y, label="Data")

x_values = np.linspace(1, 6, 100)

linear_values = np.polyval(coefficients, x_values)

plt.plot(x_values, linear_values, label="Linear Regression")

plt.scatter(query, lwr_prediction,
            label="LWR Prediction")

plt.xlabel("X")
plt.ylabel("Y")
plt.title("Locally Weighted Regression")
plt.legend()
plt.show()
```

OUTPUT:

```
=============== RESTART: E:/Machine Learning/C04-AT1 PROGRAM2.py ===
Enter query point: 4.5

Locally Weighted Regression Prediction: 8.878293882895976
Standard Linear Regression Prediction: 8.861904761904766
```



3. Write a Python program to implement a simple Radial Basis Function (RBF) network for regression or classification. Use Gaussian functions as activation units and train the model on a small dataset. Display predicted outputs and analyze how centers and widths affect performance.

PROGRAM:

```
import numpy as np

# Training data
X = np.array([1, 2, 3, 4, 5], dtype=float)
```

```
Y = np.array([2, 4, 6, 8, 10], dtype=float)

# RBF centers
centers = np.array([1, 3, 5], dtype=float)

# Width of Gaussian functions
width = 1.0

def gaussian_rbf(x, center, width):
    return np.exp(-((x - center) ** 2) /
                  (2 * width ** 2))

# Create RBF feature matrix
Phi = np.array([
    [gaussian_rbf(x, c, width) for c in centers]
    for x in X
])

# Train weights
weights = np.linalg.pinv(Phi) @ Y

def predict(x):
    features = np.array([
        gaussian_rbf(x, c, width)
        for c in centers
    ])

    return features @ weights

print("Centers:", centers)
```

```
print("Width:", width)
print("Weights:", weights)

print("\nPredicted Outputs:")

for x in X:
    prediction = predict(x)
    print("Input:", x, "Predicted:", round(prediction, 2))

# Test input
test = float(input("\nEnter a test value: "))
print("Prediction for", test, "=", round(predict(test), 2))
```

OUTPUT:

```
=============== RESTART: E:/Machine Learning/C04-AT1 PROGRAM3.py =
Centers: [1. 3. 5.]
Width: 1.0
Weights: [1.53534931 4.49871527 9.20160174]

Predicted Outputs:
Input: 1.0 Predicted: 2.15
Input: 2.0 Predicted: 3.76
Input: 3.0 Predicted: 5.95
Input: 4.0 Predicted: 8.33
Input: 5.0 Predicted: 9.81

Enter a test value: 3.5
Prediction for 3.5 = 7.02
```

4. Create a Python program that implements a basic Case-Based Learning system. Store past cases in a dataset and retrieve the most similar case for a new query using distance measures. Output the solution based on the closest match and explain similarity calculation.

PROGRAM:

```
import math

cases = [
    ([20, 80], "Low Risk"),
    ([25, 75], "Low Risk"),
    ([35, 60], "Medium Risk"),
    ([40, 55], "Medium Risk"),
    ([50, 40], "High Risk"),
    ([60, 30], "High Risk")
]

def euclidean_distance(case1, case2):
    return math.sqrt(
        sum((a - b) ** 2
            for a, b in zip(case1, case2))
    )

age = float(input("Enter age: "))
score = float(input("Enter score: "))

query = [age, score]

best_case = None
best_solution = None
min_distance = float("inf")

for features, solution in cases:

    distance = euclidean_distance(query, features)
```

```
    print("Case:", features,
          "Distance:", round(distance, 2))

    if distance < min_distance:
        min_distance = distance
        best_case = features
        best_solution = solution

print("\nMost Similar Case:", best_case)
print("Distance:", round(min_distance, 2))
print("Predicted Solution:", best_solution)
```

OUTPUT:

```
=============== RESTART: E:/Machine Learning/C04-AT1 PROGRAM4.py =
Enter age: 37
Enter score: 58
Case: [20, 80] Distance: 27.8
Case: [25, 75] Distance: 20.81
Case: [35, 60] Distance: 2.83
Case: [40, 55] Distance: 4.24
Case: [50, 40] Distance: 22.2
Case: [60, 30] Distance: 36.24

Most Similar Case: [35, 60]
Distance: 2.83
Predicted Solution: Medium Risk
```

5. Create a Python program to implement the K-Means clustering algorithm. Initialize cluster centers, assign points to clusters, and update centroids iteratively. Display final clusters and visualize them using plots. Analyze how the number of clusters and initialization methods affect clustering results.

PROGRAM:

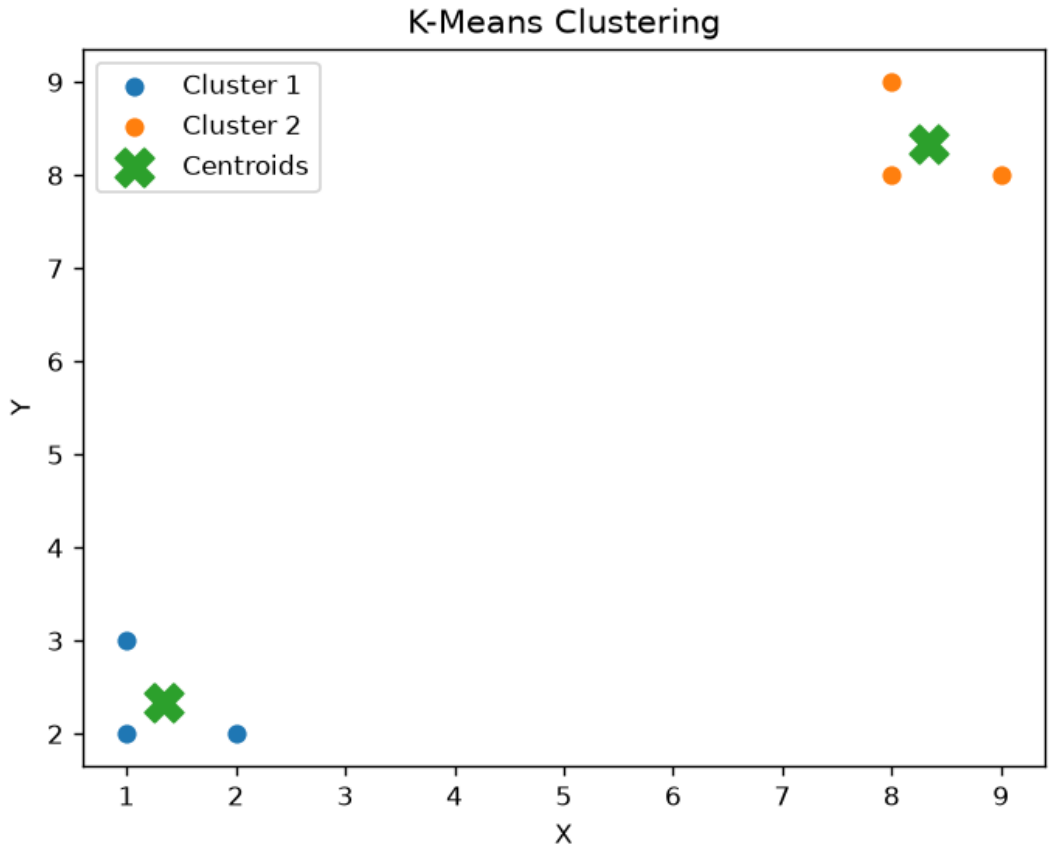
```
import numpy as np
import matplotlib.pyplot as plt

# Dataset
X = np.array([
    [1, 2],
    [1, 3],
    [2, 2],
    [8, 8],
    [9, 8],
    [8, 9]
], dtype=float)

# Number of clusters
k = int(input("Enter number of clusters K: "))

# Initial centroids
centroids = X[:k].copy()

for iteration in range(10):

    # Calculate distance from each point to each centroid
    distances = np.array([
        [np.linalg.norm(point - centroid)
         for centroid in centroids]
        for point in X
    ])

    # Assign each point to nearest centroid
```

```
    labels = np.argmin(distances, axis=1)

    # Update centroids
    new_centroids = []

    for i in range(k):
        cluster_points = X[labels == i]

        if len(cluster_points) > 0:
            new_centroids.append(
                cluster_points.mean(axis=0)
            )
        else:
            new_centroids.append(centroids[i])
    new_centroids = np.array(new_centroids)

    # Check convergence
    if np.allclose(centroids, new_centroids):
        break

    centroids = new_centroids

print("\nFinal Centroids:")
print(centroids)

print("\nClusters:")
for i in range(k):
    print("Cluster", i + 1)
    print(X[labels == i])
```

```
# Visualization
for i in range(k):
    points = X[labels == i]
    plt.scatter(points[:, 0], points[:, 1],
                label="Cluster " + str(i + 1))

plt.scatter(
    centroids[:, 0],
    centroids[:, 1],
    marker="X",
    s=200,
    label="Centroids"
)
plt.xlabel("X")
plt.ylabel("Y")
plt.title("K-Means Clustering")
plt.legend()
plt.show()
```

OUTPUT:

```
=============== RESTART: E:/Machine Learning/C04-AT1 PROGRAM5.py =
Enter number of clusters K: 2

Final Centroids:
[[1.33333333 2.33333333]
 [8.33333333 8.33333333]]

Clusters:
Cluster 1
[[1. 2.]
 [1. 3.]
 [2. 2.]]
Cluster 2
[[8. 8.]
 [9. 8.]
 [8. 9.]]
```



RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**20**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**15**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**20**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**15**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand